EngageNet

M3 Presentation - May 31, 2026

Lado Turmanidze, Luka Javakhishvili, Mariami Gadelia, Keso Chikhladze

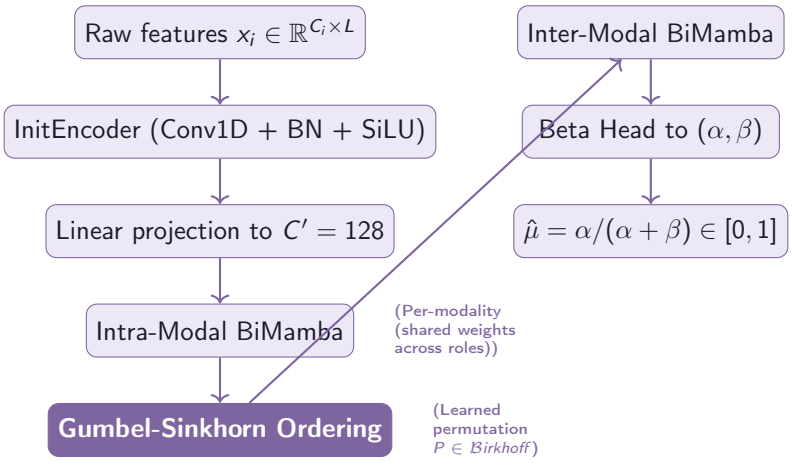
Supervisors: Dr. Philipp Müller & Beso Mikaberidze

May 31, 2026

Today's Agenda

1. **Full architecture recap** - end-to-end data flow
2. **Beta distribution justification** - why not Kumaraswamy or logit-normal?
3. **Implementation** - JAX + Flax Linen, project structure
4. **Engineering beyond the paper** - what research papers never show
5. **Evaluation metrics** - CCC and CDD
6. **Status & next steps**

Part 1: Full Architecture - Data Flow



Architecture - Key Design Choices

Selective State Space (SSM)

- Input-dependent Δ_t, B_t, C_t
- $O(\log L)$ via associative scan
- Bidirectional: forward + reversed

Inter-Modal Fusion

- Transpose: time $\leftrightarrow$ modality
- BiMamba across $\sum C_i'$ axis
- Order learned via Gumbel-Sinkhorn

Weight Sharing

Same encoder for expert & novice of the same modality $\rightarrow$ 10 encoders, not 20.

Core Modalities (4)

FAUs, gaze, pose, audio (eGeMAPS). Chosen for temporal richness & complementarity.

Part 2: Why Beta? - Comparison with Alternatives

Requirements

1. Support on $[0, 1]$ (no leakage)
2. Closed-form variance at inference
3. Closed-form KL for TTA loss
4. Differentiable for backprop

Scorecard

	Beta	Kuma.	Logit-N
Support $[0, 1]$	✓	✓	✓
Var (closed)	✓	×	×
KL (closed)	✓	×	∼
Fast inference	✓	×	×

Beta vs Kumaraswamy - The Inference Cost

Kumaraswamy variance

Requires Γ function evaluations:

$$\mathrm{Var}[Y] = b\,B\left(1+\tfrac{2}{a}, b\right) - \left[b\,B\left(1+\tfrac{1}{a}, b\right)\right]^2$$

Expensive at 25 Hz real-time streaming.

Beta variance

Pure arithmetic:

$$U(x) = \frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)}$$

No transcendental functions at inference.

Monotonic proxy: $\partial U/\partial\kappa < 0$ and $\partial H/\partial\kappa < 0 \implies$ variance ranks samples identically to differential entropy.

Beta vs Logit-Normal - The KL Problem

Logit-Normal issues

- Moments need numerical integration
- KL in $[0, 1]$ space: no closed form
- Gradients explode near boundaries

Beta KL (closed-form)

$D_{KL}(B_1 \| B_2) = \ln \frac{B_2}{B_1} + (\alpha_1 - \alpha_2)\psi(\alpha_1) + \cdots$

Exact, stable, used directly in $\mathcal{L}_{\text{mis}}$ for TTA.

Beta: transcendental cost during training (digamma in grads), free at inference.

Part 3: Implementation - JAX + Flax Linen

Why JAX?

- `jax.lax.associative_scan` for $O(\log L)$ SSM
- XLA compilation
- Functional transforms: `jit`, `grad`
- I wanted to see what the hype was about

Why Flax Linen?

- `nn.Module` with explicit params
- Kinda like functional programming (I love OCaml)
- Seamless Orbax checkpointing
- Clean separation: model vs state

All randomness controlled via `jax.random.PRNGKey(95)` with explicit key splitting.

Project Structure

src/ - Core modules

config.py	Centralised cnfg
ssm.py	JAX assoc. scan
bimamba.py	BiMambaBlock
inter_modal.py	Gumbel-Sinkhorn
beta_head.py	Beta regression
model.py	Full EngageNet
init_encoder.py	Shallow CNN
modality_frontend.py	Per-modality entry

src/ - Pipeline & eval

train.py	AdamW + early stop
tta.py	Test-time adaptation
inference.py	Multi-corpus CSV out
dataset.py	Lazy streaming loader
data_loader.py	Batch iterator
metrics.py	CCC, CDD
evaluate.py	Scoring script
aggregator.py	Overlap-add windows

Part 4: What We Added Beyond the Paper

Data engineering?

- Lazy streaming for 88+ GiB dataset (one session at a time in memory)
- Dynamic `active_modalities` filter

Reproducibility

- CLI via `EngageNetConfig.from_cli()`
- Orbax checkpoints every N epochs
- Docker containers (train + inference)

Training details

- Tau annealing: $\tau_t = \max(\tau_\infty, \tau_0 \cdot \gamma^t)$
- CCC-based validation + early stopping
- AdamW with weight decay

Inference pipeline

- Per-session processing
- Overlap-add window aggregation
- Submission CSV generation

Surgical TTA - Implementation Choices

Frozen vs updated

- **Updated:** BatchNorm, first Conv1D in InitEncoder, first Dense in inter-modal
- **Frozen:** All SSM params, Beta head, intra-modal BiMamba weights

Why surgical?

- Prevents catastrophic forgetting
- Mitigates Variance Starvation: model cannot inflate Var[y] to bypass hard samples
- Only 3–5% of total params updated

Part 5: Evaluation Metrics

CCC (primary metric)

$$\text{CCC} = \frac{2\rho\,\sigma_{\hat{y}}\sigma_y}{\sigma_{\hat{y}}^2 + \sigma_y^2 + (\mu_{\hat{y}} - \mu_y)^2}$$

Penalises both scale and location shift.
Combined: $\frac{1}{N}\sum_n \text{CCC}_n$.

CDD (fairness)

$$\text{CDD} = \frac{1}{|G|}\sum_g |\text{CCC}_g - \text{CCC}_{\text{all}}|$$

Computed per gender and per language.
Lower = more equitable.

Challenge ranking: maximise CCC while keeping CDD below threshold.

Status & Next Steps

Done ✓

- Full architecture coded
- All unit tests pass
- Docker containers ready
- Paper draft in solid shape

In Progress & Next Steps

- Using MICM compute for training
- Training on NoXi
- Hyperparameter tuning?
- TTA evaluation on NoXi+J

Questions?